

Networking Use Cases

Session Timeout

Iteration: 1, last modification: March 07, 2025

Description	Start a timer that count down the player's inactive time and automatically
Primary Actor	Player
Goal in Context	Ensure inactive players are automatically disconnected to free up server resources and maintain a smooth multiplayer experience.
Preconditions	1. The player is connected to the server and in an active game session. 2. A session timeout policy is defined (e.g., disconnect after 5 minutes of inactivity).
Trigger	The player has been inactive for a predefined period.
Scenario	The system does not distinguish between a player intentionally quitting vs. accidental disconnections due to internet loss or misclicks. Suggestion: could allow players a window of time to reconnect even through manual accidental disconnection. 1. The player stops interacting with the game (no moves, chat, or other actions). 2. The server starts a countdown for the session timeout. 3. The server sends a warning message to the player about an impending connection. 4. If no response is received, the server disconnects the player from the game. 5. The server updates the game state and notifies the remaining players.
Exceptions	1. If the player resumes activity before the timeout, the countdown is reset. 2. If the disconnection was due to network issues, an automatic reconnection attempt is made.
Priority	High (ensures smooth server performance and fair gameplay).
Frequency of Use	Occasionally (depends on player inactivity)
Channel to Actor	Inconsistent formatting, fix formatting for clarity 1. : Manages the session timeout and disconnection process. Other Players: Receive a notification that the player has been removed

Secondary Actors	GameServerManager, Other players
Open Issues	1. Should disconnected players be allowed to rejoin the same session? 1. How long should the server wait before removing inactive players?

Inconsistent formatting, fix
formatting for clarity



Network Connection

Iteration: 1, last modification: March 7, 2025

Description	
Primary Actor	Player
Goal in Context	Ensure the player can establish and maintain a stable connection with the game server for real-time interactions.
Preconditions	• The player has a stable internet connection. • The game client is launched and ready to connect
Trigger	The player starts the game and attempts to connect to the server.
Scenario	The player opens the game and selects "Connect to Server." The game client sends a connection request to the server. The server authenticates the request and establishes a session. If successful, the server sends a confirmation message. The player is now connected and can proceed to matchmaking or gameplay.
Exceptions	• If the connection request fails: ◦ The player receives an error message (e.g., "Server Unreachable") ◦ The system may retry the connection automatically • If the player loses connection mid-game: ◦ The system attempts reconnection within a set time. ◦ If unsuccessful, the player is marked as disconnected in the game session.
Priority	Critical (network stability is essential for multiplayer functionality).
Frequency of Use	Every time a player starts a session.
Channel to Actor	In-game UI notifications (connection status, error messages).
Secondary Actors	GameServerManager, Authentication System.
Open Issues	• How many reconnection attempts should be allowed before disconnecting the player?

- | | |
|--|--|
| | <ul style="list-style-type: none">• Should the game allow offline mode if the server is unreachable? |
|--|--|

Update States

Iteration: 1, last modification: March 7, 2025

Error in formatting, fix
formatting for clarity

Description	
Primary Actor	Player
Goal in Context	6. Update the states of the game synchronously 7. Enhance the user experience with seamless transitions/ changes 8. Handle delays accordingly and minimize delays
Preconditions	1. The game is running and the network is established 2. The player has a stable internet connection
Trigger	1. The player makes a move in the game 2. The players interact with each other in the game
Scenario 1	1. The player makes a valid move(change) in the game. 2. This triggers the “Update states” use case. 3. The game server updates the changes in the interface of the other players.
Scenario 2 It would be better to use different update functions for game and chat updates as in this case, the entire game state would update for every chat message, causing immense server load.	1. The players send messages to each other through in-game chat. 2. This triggers the “Update states” use case. 3. The game server updates the messages in the interface of all players through web socket servers.
Exceptions Missing rollback mechanism for failed updates. If an update fails mid-process, how does the system recover? Suggestion: Could return to the previous game state if the update fails.	1. If the connection of a player fails, it will cause some delays in updating the states, and the player will potentially leave the server temporarily 2. If the server is packed, the server may face some issues with updating the states
Priority	Critical. This feature is the most fundamental aspect of a multiplayer game
Frequency of Use	Multiple Times

Channel to Actor	GUI, Game server
Secondary Actors	Game server system
Open Issues	1. How should the system handle the connection problems so that the delays in the game are minimized? 2. Should the system save the chat history?

Retrieve Data

Iteration: 1, last modification: March 7, 2025

Description	
Primary Actor	Player
Goal in Context	1. Enable authentication of user information and retrieving the data for leaderboard and matchmaking processes 2. Save the current state of the game to continue from the saved state 3. Update miscellaneous information and store properly
Preconditions	1. The game is running and the network is established 2. The database is running properly 3. The player has a stable internet connection 4. The request is valid
Trigger	1. The player log into the game 2. The player saves the current state of the game 3. The player looks at the real-time leaderboard 4. The player updates user information 5. The player looks for the players in the network to play with
Scenario 1	1. The player enters the game 2. The player log into the account by providing the username and the password 3. The player clicks the button to authenticate 4. The system retrieves data from the database and check if the information matches or not 5. The system approves the player if the information matches
Scenario 2	1. The player wants to save the current state of the game 2. The player clicks "Save" button 3. The system accesses the database and updates the information

	The data is updated and ready to be retrieve to resume the saved game
Scenario 3	- The player clicks the button to look at the leaderboard - The system accesses the database to retrieve the corresponding data. - The leaderboard is shown to the player through GUI
Scenario 4	- The player wants to update the user information - The player makes changes to the information and clicks 'Update' - The system accesses the database and updates the information
Scenario 5	- The player starts a game and looks for the players to play with - The system accesses the database to find the players who are active now and have the same ranking as the player - The system adds those players to the lobby and the game is ready to begin
Exceptions	- Returns an error message if the data requested is not in the database. - If connection to the database is not successful, accessing the database fails. - Wrong data may be retrieved if the database is corrupted.
Priority	Critical. This feature is the most fundamental aspect of a multiplayer game
Frequency of Use	Multiple times
Channel to Actor	GUI, Game server
Secondary Actors	WebSocket server
Open Issues	- What security measures should be added to the system to ensure the integrity of the database? - What other information should be allowed to store in the database to enhance the experience of the user?

Verify Data

Iteration: 1, last modification: March 7, 2025

Major formatting error, cannot see data



Description	
Primary Actor	Player
Goal in Context	Authorize the log-in process to make sure that only the intended user can access the account
Preconditions	1. The game is running and the network is established 2. The database is running properly 3. The player has a stable internet connection
Trigger	1. The player clicks ‘Log-in’ button to access the account
Scenario	1. The player tries to log into the account by providing necessary personal information 2. The system retrieves the data from the database corresponding to the provided information 3. The system compares the two data and decides to authorize the player
Exceptions	1. The log-in attempt is rejected if the information does not match 2. If there is no existing data regarding the player, the player can sign up for a new account 3. The log-in process will fail if the database is corrupted
Priority	Critical. This feature is the most fundamental aspect of a multiplayer game
Frequency of Use	Multiple times
Channel to Actor	GUI, Game server
Secondary Actors	WebSocket server
Open Issues	1. What security measures should be added to the system to ensure the integrity of the data? 2. What action should be taken if the player fails 5 times to log in? 3. What account recovery methods should be in the system?

Update Data

Iteration: 1, last modification: March 7, 2025

Description	
Primary Actor	Player
Goal in Context	Update the data in the database
Preconditions	1. The game is running and the network is established 2. The database is running properly 3. The player has a stable internet connection 4. The player is already logged into the account successfully
Trigger	1. The player clicks 'Update' button to update the user information 2. The player clicks 'Save' button to save the current states of the game 3. Formatting error, empty space
Scenario 1	1. The player makes some changes to the user information 2. The player clicks 'Update' button to update the information 3. The system updates the user information in the database
Scenario 2	1. The player wants to save the current state of the game to resume at where it left off 2. The player clicks 'Save' button to save the states 3. The system updates the saved states in the database
Exceptions How would you handle update fails?	1. No update is done if the player's information has no changes. 2. Update is rejected if the information is in an unacceptable format 3. If the database fails to connect, update cannot be done
Priority	Critical. This feature is the most fundamental aspect of a multiplayer game
Frequency of Use	Multiple times
Channel to Actor	GUI, Game server
Secondary Actors	WebSocket server

Open Issues	1. What happens if an update fails midway? 2. Should changes be logged for tracking and compliance purposes?
--------------------	--

Retrieve Data

Iteration: 1, last modification: March 7, 2025

Description	
Primary Actor	Player
Goal in Context	Retrieve the data from the database for various purposes
Preconditions	1. The game is running and the network is established 2. The database is running properly 3. The player has a stable internet connection 1. The player is already logged into the account successfully
Trigger	The player make actions which requires the system to retrieve data from the database
Scenario 1	1. The player logs into the account 2. The player provides the log-in credentials 3. The system retrieves data from the database to verify
Scenario 2	1. The player opens the leaderboard 2. The system retrieves data regarding the leaderboard from the database
Scenario 3	1. The player starts a game and wait for other players to join 2. The system accesses the database to find the players whose ranks match with the player who is in the game lobby
Exceptions	1. The requested data does not exist in the database 2. The user lacks the necessary permissions to retrieve the data 3. The request contains incorrect or malformed filters
Priority	Critical. This feature is the most fundamental aspect of a multiplayer game
Frequency of Use	Multiple times
Channel to Actor	GUI, Game server
Secondary Actors	WebSocket server

Consider adding an exception for handling database timeouts or connection failures to prevent data retrieval failures from crashing the system.

Open Issues	1. Should frequently accessed data be cached to improve performance? 2. Should data retrieval be logged or monitored for unauthorized access?
--------------------	---

In-Game Chat System

Iteration: 1, last modification: March 7, 2025

Description	
Primary Actor	Player
Goal in Context	Enable real-time communication between players in a game session. Ensure chat messages are delivered efficiently and securely.
Preconditions	1. Players are connected to the game server 2. The game session is active 3. The chat system is enabled
Trigger	A player types and sends a message in the in-game chat
Scenario	1. The player enters a message in the chat box 2. The client sends the message to the GameServerManager using WebSockets. 3. The GameServerManager processes and filters the message (e.g., checks for inappropriate content). 4. The server broadcasts the validated message to all players in the session. 5. Each player's client receives the message and displays it in the chat window. 6.
Exceptions	1. Network failure: The message is delayed or not sent. 2. Server overload: Chat messages experience a delay. 2. Inappropriate content detected: The message is blocked or censored.
Priority	High (important for player communication and engagement)
Frequency of Use	Multiple times per game session.
Channel to Actor	In-game chat UI.
Secondary Actors	GameServerManager, Other players
Open Issues	1. Should chat history be stored after the game ends? 2. Should players be able to mute other players? 3. How should spam messages be handled?

Consider adding a spam detection mechanism to prevent players from flooding the chat with messages

Inconsistent formatting

If a player disconnects and reconnects, should they receive missed messages? Define whether chat history is temporarily stored for reconnecting users.